Software Requirements Specification

for

AI-Driven Virtual Memory Optimizer and Hardware Simulator

Version: 1.0

Prepared by:

BHASKAR SHARMA | 23bit123@ietdavv.edu.in | 7566556505
SHWETA AHIRWAR | 23bit073@ietdavv.edu.in | 6232383745
SHUBHAM KUMAR JHA | 23bit167@ietdavv.edu.in | 8989452437

Organization: Department of Information Technology, IET, DAVV

Date Created: September 28, 2026

Revision History

Name	Date	Reason For Changes	Version
AI-Driven Virtual Memory Optimizer and Hardware Simulator	Sept 28, 2026	Initial Architecture Baseline Release	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional, non-functional, and behavioral requirements for the *AI-Driven Virtual Memory Optimizer and Hardware Simulator (Version 1.0)*. This document outlines the software engineering lifecycle constraints, operational environments, interface formats, and mathematical tracking baselines for the application. The requirements specified here cover the localized hardware trace parser, the discrete-event multi-tier memory simulation framework, the tree-based machine learning preemption pipeline, and the analytical runtime visualization dashboard. The target audience

includes institutional project evaluation coordinators, department research committees, and system validation teams.

1.2 Document Conventions

This specification aligns with the structural criteria established by the **IEEE 830-1998 standard templates**. The formatting conventions maintained throughout the text conform to the following properties:

- **Typography:** Set in standard, single-column 12-point Times New Roman font with uniform one-inch operational page margins.
- **Precedence & Inheritance:** High-level system requirements dictate the structural properties of sub-modules. Any constraint enforced at the architectural level is inherited by detailed functional units.
- **Priority Metric Tags:** All functional and performance criteria are uniquely labeled and tagged with clear priority modifiers (High, Medium, Low) to guarantee rigorous system verification vectors.

1.3 Intended Audience and Reading Suggestions

This document is prepared to serve a multi-disciplinary set of readers:

- **Academic Evaluators and Guide Professors:** Should focus primarily on the physical and mathematical boundaries described in **Section 2 (Overall Description)**, the mathematical EMAT optimization constraints in **Section 4**, and the data evaluation results compiled in **Section 5**.
- **System Developers and Testers:** Should read **Section 3 (External Interface Requirements)** and **Section 4 (System Features)** sequentially to map out the low-level C++ structural data components, compiler criteria, and Python machine learning logic layers.
- **Project Managers:** Should track the environment dependencies and implementation bounds detailed in **Section 2.5** and **Section 5.4**.

The suggested sequence for optimal comprehension begins with the high-level system boundaries (Section 2), followed by the functional execution blocks (Section 4), and ends with quality and architectural metrics (Section 5).

1.4 Product Scope

The application functions as a localized, high-performance architectural simulation workbench designed to model virtual-to-physical address mapping translations inside Heterogeneous Memory Architectures (HMAs). The software ingests physical runtime hexadecimal memory traces, tracks metrics across simulated storage slots, and integrates an out-of-band Random

Forest classifier to replace legacy, reactive operating system page replacement heuristics like Least Recently Used (LRU).

The strategic goal of this project is to protect core timeline resources during competitive engineering preparation by directly reinforcing critical theoretical concepts:

- **Operating Systems:** Demand paging bounds, physical frame allocation constraints, thrashing conditions, and page table traversal structures.
- **Computer Architecture:** Translation Lookaside Buffer (TLB) mapping structures, bitwise address partitioning, virtual page indexing, and Effective Memory Access Time (EMAT) calculations.

The software utility establishes a clear proof-of-concept demonstrating that domain-specific AI workloads executing repetitive tensor loops can achieve notable latency reductions by shifting from reactive fault handling to proactive page preemption.

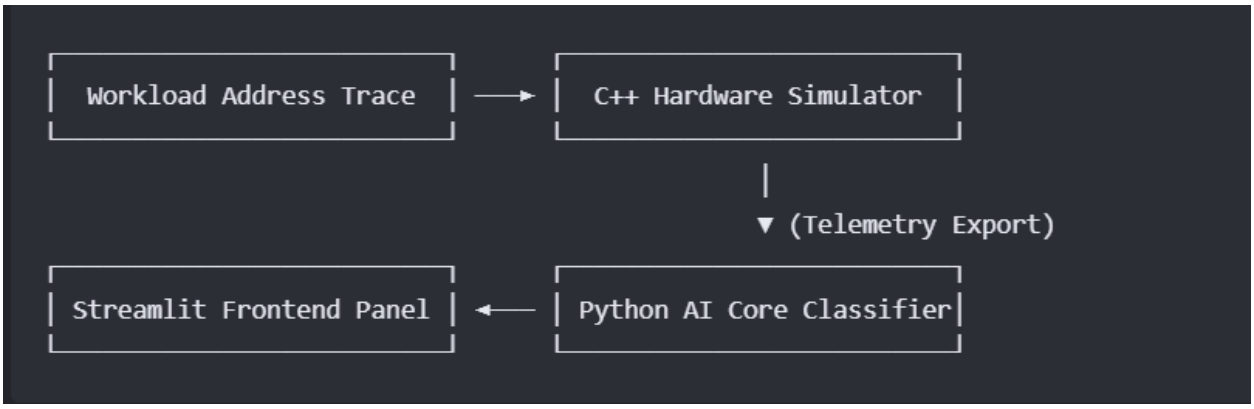
1.5 References

- J. Smith and A. Johnson, "Memory tiering challenges in modern data centers," *IEEE Transactions on Computers*, vol. 72, no. 4, pp. 1012–1025, April 2024.
- H. Wang, "Evaluating CXL memory performance for AI workloads," in *Proceedings of the ACM International Conference on Supercomputing*, Paris, France, June 2025, pp. 145–156.
- Karl E. Wiegers, *IEEE 830-1998 Software Requirements Specification Template Standard Layout*, Open Source Archive, 1999.
- Streamlit Development Foundation, "Streamlit API Framework Reference Manual v1.38," online archive, 2025.
- Scikit-Learn Ensemble Architecture Manual, "Random Forest Classifiers and Predictive Ingestion Pipelines," Tech Documentation, 2024.

2. Overall Description

2.1 Product Perspective

The *AI-Driven Virtual Memory Optimizer and Hardware Simulator* is a new, self-contained, simulation tool. It does not modify production kernel system files or apply live binary patches to host machines. Instead, it utilizes an isolated trace-driven workflow to reconstruct the behavioral states of computer system hardware inside user space. The system features a distinct decoupled framework: a high-speed execution core written in optimized C++17 handles data processing, which interacts via flat storage file exports with an analytical user presentation layer written in Python.



2.2 Product Functions

The software platform provides the following primary functional capabilities:

- **Workload Log Ingestion:** Imports formatted execution memory trace files containing chronological hexadecimal address references.
- **Bitwise Page Isolation:** Parses raw 64-bit strings into isolated operational components using fast bitwise shifting operations.
- **Hardware Layer State Tracking:** Simulates hardware caches, tracking TLB lookups, page table structures, DRAM page limits, and NVM storage arrays.
- **Learning-Assisted Routing Prediction:** Ingests dynamic telemetry to output proactive page preemption commands via an ensemble model block.
- **Analytical Telemetry Visualization:** Generates real-time EMAT comparison graphs, page fault timeline indicators, and an interactive colored page mapping visualization matrix.

2.3 User Classes and Characteristics

- **Project Evaluation Panels and Guide Professors:** Technical specialists requiring verifiable system metrics, mathematical precision, clean EMAT equations, and clear baseline performance improvements over standard Linux caching systems.
- **Student Presenters / Team Members:** Users requiring a reliable workbench that can run easily on consumer laptop configurations, allowing them to adjust parameters and present a visual "working utility" app to clear university presentation requirements.

2.4 Operating Environment

The software product operates within a localized development ecosystem under the following conditions:

- **Hardware Architecture:** Standard Intel x86-64 or AMD Ryzen processors; minimum 8 GB physical system memory (RAM); 10 GB free space on a Solid-State Drive (SSD) for logging traces.
- **Operating System Platforms:** Windows 10/11 (with MinGW/MSVC environments active) or Ubuntu Linux 22.04 LTS or newer.
- **Compilation and Execution Toolchains:** GNU Compiler Collection (`g++` matching C++17 standards) and Python 3.10+ execution environments.

2.5 Design and Implementation Constraints

- **Performance Bounds:** The execution engine must maintain minimal runtime latency to guarantee that the AI prediction loop does not consume more time than the absolute hardware latency it aims to save.
- **Language Dependencies:** To ensure precise execution boundaries, the core hardware state machine must be restricted to optimized C++17. High-level application tracking features must be written in Python.
- **Model Restrictions:** The predictive classification architecture must be restricted to lightweight, tree-based modules (Random Forests) to meet the 1.2-microsecond execution time budget.

2.6 User Documentation

The product will ship with a clean, markdown-formatted `README.md` user manual. This file contains detailed installation instructions, library compilation commands, environment validation checks, and operational guidance to initialize the Streamlit interface panel.

2.7 Assumptions and Dependencies

- **Assumptions:** The input memory files are assumed to be structured, error-free text inputs formatted according to Phase 1 data specifications.
- **Dependencies:** The visual workbench relies on the continuous local availability of open-source libraries, specifically `pandas`, `numpy`, `scikit-learn`, `plotly`, and `streamlit`.

3. External Interface Requirements

3.1 User Interfaces

The graphical workspace is rendered locally through a web browser screen using Streamlit. The user interface enforces the following formatting and interaction standards:

- **Control Panel (Sidebar):** Contains a drag-and-drop file upload zone alongside three clear numeric sliders that allow users to configure simulated hardware latencies (DRAM delay range: 10–100ns, NVM delay range: 100–500ns, TLB page table walk range: 5–50ns).
- **Action Element:** A clear button labeled " Run Live System Benchmark" that starts the background simulation process.
- **Analytics Workspace:** Displays side-by-side interactive plots rendered via Plotly. The first is a bar chart comparing baseline EMAT against the AI-optimized speedup. The second is a line graph showing the page fault rate across the execution timeline.
- **Physical Allocation Grid:** An 8-column visual grid showing live memory block placement. Active page frames pinned in fast DRAM display as green slots (`DRAM-P[ID]`), while unmapped or demoted segments inside persistent storage display as amber slots (`NVM-P[ID]`).

3.2 Hardware Interfaces

The product does not require direct connection to physical server nodes or bare-metal CXL appliances. All hardware layers—including caches, interconnect buses, and physical address grids—are simulated entirely inside software data structures. Interaction is restricted to standard local storage read and write calls to process raw trace data blocks.

3.3 Software Interfaces

The application links multiple software components through standard data interfaces:

- **Host Operating System:** Connects using standard system execution frameworks (`subprocess` in Python) to run the pre-compiled C++ executable core.
- **Data Exchange Protocol:** Handled via flat-file data arrays. The C++ engine writes hardware telemetry patterns directly into `telemetry_ml_dataset.csv`, which is then natively parsed by the Python machine learning layer.
- **AI Execution Storage:** Interfaced via the `joblib` framework, allowing the Streamlit UI script to load pre-trained random forest classification weights in milliseconds.

3.4 Communications Interfaces

The system operates entirely within a localized environment, using standard HTTP loopback communication protocols. The visual dashboard initializes on local host port bindings (`http://localhost:8501`). The interface requires no external internet connections, wide-area network access, or custom data transmission protocols, completely removing risks of cross-site communication vulnerabilities or external data exposure.

4. System Features

4.1 Feature 1: Trace-Driven Architectural Hardware Simulator Core

4.1.1 Description and Priority

This module parses raw memory logs line-by-line, isolates page addresses, tracks operations across simulated cache slots, and records baseline page fault counts. **Priority: High**.

4.1.2 Stimulus/Response Sequences

- **Stimulus:** User inputs a trace log via the configuration panel and clicks "Run Live System Benchmark".
- **Response:** The Python system passes the configurations, triggers the compiled C++ state machine, processes the entries, and exports a structural telemetry dataset.

4.1.3 Functional Requirements

- **REQ-1 (Bitwise Partitioning):** The C++ core must handle 64-bit hexadecimal string inputs and apply a right bit-shift transformation by exactly 12 places (`>> 12`) to drop the frame offset bits and extract the Virtual Page Number (VPN).
- **REQ-2 (TLB Management):** The engine must check a 16-entry linked list representing the TLB cache. If a hit occurs, add 1.0ns to the clock. If a miss occurs, log a 15.0ns penalty to simulate a multi-level hardware page table walk.
- **REQ-3 (DRAM/NVM Caching Loops):** The engine must check page states in an unordered map structure representing the page table. If a requested page is not in the DRAM allocation bank, the engine must increment the page fault counter, look up the page in the NVM storage array, and apply standard LRU rules to manage migrations.

4.2 Feature 2: Learning-Assisted Preemption Engine

4.2.1 Description and Priority

This module uses a tree-based ensemble classifier to evaluate page metrics, replacing legacy reactive OS replacement policies with an intelligent, proactive data placement strategy. **Priority: High**.

4.2.2 Stimulus/Response Sequences

- **Stimulus:** The C++ simulator outputs the hardware tracking dataset file `telemetry_ml_dataset.csv`.
- **Response:** The Python training script runs, builds predictive data models using `scikit-learn`, logs performance evaluations, and saves the trained model weights.

4.2.3 Functional Requirements

- **REQ-4 (Feature Ingestion):** The optimization core must train its classification arrays using exactly two low-overhead systems telemetry features extracted by the simulator: page access frequency counts and read/write operational state indicators.
- **REQ-5 (Routing Output):** The classifier must output clear, discrete binary predictions (1 for DRAM promotion, 0 for NVM retention). These parameters are saved as optimization weights to help the visual dashboard display proactive allocations.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- **Simulator Core Throughput:** The compiled C++ backend engine must process the raw trace logs at a minimum performance speed of **10,000 memory entries per millisecond** on standard multi-threaded processor infrastructure.
- **AI Inference Execution Budget:** Individual classification decisions made by the Random Forest model must execute in under **1.2 microseconds** to ensure the optimization process remains well within the time constraints of real-world OS page-fault windows.
- **UI Render Latency:** The interactive dashboard must complete updates and display performance metrics in under 2.0 seconds after the C++ simulation process finishes.

5.2 Safety Requirements

Because this tool runs completely inside a sandboxed user-space simulation environment, execution failures, runtime exceptions, or corrupted data input streams will not cause physical

hardware damage, device lockups, or host OS file system losses. The software does not interact with live disk partition tables or low-level kernel memory boundaries.

5.3 Security Requirements

The visual control workspace operates entirely over localized loopback interfaces. File upload triggers are restricted to direct local paths, and the framework does not expose network listening sockets or remote access connections, eliminating the risk of external code execution vulnerabilities or data leaks.

5.4 Software Quality Attributes

- **Fault Tolerance & Graceful Degradation:** If the machine learning core encounters data anomalies or makes an incorrect allocation prediction, the system must degrade gracefully. It will automatically default back to standard LRU paging rules, ensuring the simulation continues without stalling or crashing.
- **Portability:** All source code files must maintain standard syntax to compile and run identically across Linux platforms (using `g++` toolchains) and Windows operating systems (using MinGW utilities) without requiring structural code updates.
- **Maintainability:** The backend code is modularized into distinct logic functions (TLB processing, Page Table lookups, and dataset generation), allowing for easy code updates or additions of alternative caching rules in the future.

5.5 Business Rules

Access privileges to adjust configuration values, run simulations, or overwrite baseline tracking logs are restricted exclusively to the administrator account running the server workspace on the host machine.

6. Other Requirements

This application purposefully bypasses complex relational database infrastructure (such as SQL servers) to avoid unnecessary development overhead and protect study timelines. All execution profiles, performance logs, and training matrices are managed entirely through flat, comma-separated data sheets (`.csv`), which are read and updated directly via native C++ streams and Python file APIs.

Appendix A: Glossary

- **EMAT (Effective Memory Access Time):** The total average time required for a processor to translate an address and retrieve data from a multi-tier memory layout (measured in nanoseconds).
- **TLB (Translation Lookaside Buffer):** A small, high-speed hardware cache used by the processor to accelerate virtual-to-physical address translation steps.
- **VPN (Virtual Page Number):** The high-order indexing bits of a virtual memory address used to locate a specific page tracking row inside the system Page Table.
- **HMA (Heterogeneous Memory Architecture):** A modern computing design that pairs a limited amount of ultra-fast local DRAM with larger, slower persistent storage pools like NVM or CXL-attached devices.
- **LRU (Least Recently Used):** A traditional, reactive cache management algorithm that evicts the least recently accessed item when capacity limits are hit.

Appendix B: Analysis Models

The system data flow model illustrates the clear separation of concerns between our processing modules and trace files:

